

Métodos Computacionais em Física de Altas Energias

Python - Parte 1

Marco Dias

15 de agosto de 2026

Python é uma linguagem de programação dinâmica usada em uma ampla variedade de domínios de aplicação.

- Linguagem orientada a objetos, interpretada e interativa
- Possui módulos, classes, exceções, tipos de dados dinâmicos de alto nível e tipagem dinâmica
- Interfaces para muitas chamadas de sistema e bibliotecas
- Novos módulos podem ser escritos em C, C++ ou outras linguagens

Estilo de codificação recomendado:

- Indentação de 4 espaços
- Sem uso de tabs

Os arquivos de configuração do CMS (CMSSW) usam Python.

Mais informações: <http://www.python.org/>

Exemplo Básico em Python

```
1 #!/usr/bin/env python
2 import sys, os, re
3 root_re = re.compile("(?P<rootfile>([~/]*)\\.root")
4 def main():
5     path = os.path.abspath(sys.argv[1])
6     dirs = execute("ls %s" % path)
7     for dir in dirs:
8         dir = os.path.join(path, dir)
9         if os.path.isdir(dir):
10             subdirs = execute("ls %s" % dir)
11             if subdirs.count("res") == 1:
12                 dir = os.path.join(dir, "res")
13                 files = execute("ls %s" % dir)
14                 for file in files:
15                     match = root_re.search(file)
16                     if match:
17                         print("file:" + os.path.join(dir, file))
```

Exemplo Básico em Python

```
1 def execute(cmd):  
2     f = os.popen(cmd)  
3     ret = []  
4     for line in f:  
5         ret.append(line.replace("\n", ""))  
6     f.close()  
7     return ret  
8  
9 if __name__ == "__main__":  
10     main()
```

Parâmetros de Linha de Comando

Para parâmetros mais complexos, use `optparse` ou `argparse` em vez de `sys.argv`.

```
1 from optparse import OptionParser
2
3 parser = OptionParser()
4 parser.add_option('-f', '--file', dest='filename')
5 (options, args) = parser.parse_args()
6
7 print('options', options)
8 print('args', args)
9 print(options.filename)
```

Saída:

```
1 $ python example1.py -f TEST1 TEST2
2 options {'filename': 'TEST1'}
3 args ['TEST2']
4 TEST1
```

Tipos e Opções no optparse

Tipos suportados: string, int, float

```
1 parser.add_option('-n', type='int', dest='number')
2 myint = options.number
```

Se nenhum tipo for especificado, assume-se string. **Booleanos:**

```
1 parser.add_option('-v', action='store_true', dest='verbose')
2 # -v define options.verbose = True
```

Valores padrão:

```
1 parser.add_option('-v', action='store_true',
2                   dest='verbose', default=True)
```

Ajuda automática:

```
1 parser.add_option('-f', '--file', dest='filename',
2                   help='Input filename')
```

O módulo `re` fornece padrões de expressões regulares no estilo Perl.

- Usado para casar (`match`), modificar ou dividir strings
- A maioria dos caracteres casam com eles mesmos
- Caracteres especiais (metacaracteres) precisam ser escapados com `\`

Expressões Regulares (RE) - Introdução

Metacharacter	Description	Example Pattern	Matches
.	Matches any character except a newline.	a.b	"axb", "a3b", "a b"
^	Matches the start of the string.	^Hello	"Hello world"(but not "Say Hello")
\$	Matches the end of the string.	world\$	"Hello world"(but not "world tour")
*	Matches zero or more repetitions.	abc*	"ab", "abc", "abcc"
+	Matches one or more repetitions.	abc+	"abc", "abcc"(does not match "ab")
?	Matches zero or one repetition.	abc?	"ab"or "abc"
{}	Matches a specified number of repetitions.	a{3} or a{1,3}	"aaa"or variations with 1 to 3 "a"s
[]	Defines a set/class of characters.	[a-z]	Any lowercase letter from a to z
	Acts as an OR operator.	cat dog	Matches "cat"or "dog"
()	Groups patterns and captures text.	(abc)+	Matches "abc", "abcabc", etc.
\	Escapes metacharacters or signals special sequences.	\. or \d	Matches a literal . or any single digit

Expressões Regulares - Exemplos

```
1 import re
2 # Compila a expressao regular para reuso
3 root_re = re.compile(r'(?P<filename>(\S+))\.root$')
4 names = ['test.root', '/a/b/c/test.root', 'root.txt']
5
6 for name in names:
7     match = root_re.search(name)
8     print(name)
9     if match:
10         print(' Match found:', match.group('filename'))
11     else:
12         print(' ', name, 'did not match')
```

Saída:

```
1 test.root
2   Match found: test
3 /a/b/c/test.root
4   Match found: /a/b/c/test
5 root.txt
6   root.txt did not match
```

Para obter apenas o nome do arquivo (sem caminho):

```
1 re.compile(r'(?P<filename>[~/]\S+)\.root$')
```

Explicação da Expressão Regular

RE: '(?P<filename>[/]\S+)\.root\$'

- (?P<filename>...) - Agrupamento nomeado para fácil acesso
- [/\]\S+ - Dentro do grupo:
 - [/\] - Não começa com '/'
 - \S+ - Um ou mais caracteres que não são espaço
- \.root - Ponto literal seguido de 'root'
- \$ - A string deve terminar aqui

Exemplo prático: Selecionar arquivos específicos do CMSSW

```
1 # Arquivos tau_99_1_zRD.root
2 re.compile(r'tau_\d+_\d+\.root')
3
4 # Arquivos tau-highpurity_99_1_zRD.root
5 re.compile(r'tau-highpurity_\d+_\d+\.root')
```

RE - Exemplo com Dados

Extrair valor do parâmetro 'dataVersion' do arquivo crab.log

String de exemplo:

```
1 '-CMSSW.pycfg_params=dataVersion=74Xdata:energy=8:trigger=
   HLT_IsoMu15_v7:runOnCrab=1'
```

Expressão regular:

```
1 dataVersion=(?P<dataVersion>\S+):
```

Por que o '?' é importante?

- Por padrão, a RE é **literal**
- Quer selecionar `74Xdata:energy=8:trigger=HLT_IsoMu15_v7`
- O '?' torna a RE **não-literal**
- Seleciona apenas até o primeiro ':'

Comparação:

literal (+)	Não-literal (+?)
74Xdata:energy=8:trigger=HLT_IsoMu15_v7	74Xdata

Manipulação de Caminhos com os.path

os.path é uma ferramenta poderosa para manipulação de caminhos.

```
1 import os
2 # Juntar caminhos
3 joined_path = os.path.join(path1, path2, path3)
4 # Nome do arquivo
5 filename = os.path.basename(path_to_file)
6 # Verificar se existe
7 if os.path.exists(mypath):
8     # Diretorio do arquivo
9     dir = os.path.dirname(path_to_file)
10 # Caminho absoluto
11 abs_path = os.path.abspath("../../somepath")
12 # PWD atual
13 current_dir = os.getcwd()
```

Estruturas de Dados em Python

Listas:

```
1 l = ['a', 'b', 'c']  
2 nl = [['a', 'b', 'c'], ['d', 'e', 'f']] # Lista aninhada (matriz)
```

Tuplas: (imutáveis)

```
1 t = 12345, 54321, 'hello'
```

Conjuntos (Sets): (sem elementos duplicados)

```
1 s = set(['a', 'b', 'c'])
```

Dicionários: (pares chave:valor)

```
1 d = {'first': 12, 'second': 34}
```

Estruturas de Dados em Python

```
1 numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2 # Pega o 'x' para cada 'x' na lista se o 'x' for par
3 pares = [x for x in numeros if x % 2 == 0]
4 print(pares) # Resultado: [2, 4, 6, 8, 10]
```

Classes:

```
1 class MyClass:
2     def __init__(self, var1, var2):
3         self.var1 = var1
4         self.var2 = var2
5
6 class MyClass(BaseClass): # Heranca
7     pass
```

Encapsulamento em Python

Python protege membros ocultos alterando internamente o nome para incluir o nome da classe.

```
1 class JustCounter:
2     __secretCount = 0
3
4     def count(self):
5         self.__secretCount += 1
6         print(self.__secretCount)
7
8 counter = JustCounter()
9 counter.count()    # 1
10 counter.count()   # 2
```


Encapsulamento em Python

```
1 # Isso NAO funciona:
2 print(counter.__secretCount)
3 # AttributeError: JustCounter instance has no attribute '
   __secretCount'
4
5 # Isso funciona (acesso interno):
6 print(counter._JustCounter__secretCount)    # 2
```

Observação: O nome é alterado para `_className__attrName`.

O módulo sys fornece informações sobre constantes, funções e métodos do interpretador Python.

- Argumentos de linha de comando: `sys.argv`
- Saída padrão: `sys.stdout`, `sys.stderr`, `sys.stdin`
- Caminhos de módulos: `sys.path`

Redirecionamento de saída:

```
1 import sys
2
3 print('Coming through stdout')
4 save_stdout = sys.stdout
5 fh = open('test.txt', 'w')
6 sys.stdout = fh
7 print('This line goes to test.txt')
8 # Retornar ao normal:
9 sys.stdout = save_stdout
10 fh.close()
```

NumPy - Computação Numérica

Pacote separado para computação científica em Python. Deve ser instalado antes do uso.

Recursos:

- Arrays multidimensionais
- Álgebra linear
- Transformada de Fourier
- Números aleatórios
- Ferramentas para integrar código C/C++/Fortran

```
1 import numpy
2
3 # Array
4 a = numpy.array([1, 2, 3])
5 print(a, a.dtype)
```

```
1 # Matriz
2 m2x2 = numpy.array([(1.0, 0.0), (0.0, 1.0)])
3 print(m2x2, m2x2.dtype)
4 print('Traco:', numpy.trace(m2x2))
5 print('Autovalores:', numpy.linalg.eig(m2x2))
6 print('Norma:', numpy.linalg.norm(m2x2))
7
8 # Estatísticas
9 data = [1, 2, 3, 4]
10 print('Media:', numpy.mean(data))
11 print('Desvio padrao:', numpy.std(data))
```

Operador Lambda

Python tem duas ferramentas para construir funções: `def` e `lambda`.
`lambda` permite definir mini-funções de uma linha em tempo real.

```
1 # Usando def
2 def square(number):
3     return number * number
4 # Usando lambda
5 lsquare = lambda x: x * x
6 a = 5
7 print(a, 'sq=', square(a), '(function def)')
8 print(a, 'sq=', lsquare(a), '(lambda function)')
```

Quando usar `lambda`:

- Funções simples de uma linha
- Com funções como `map()`, `filter()`, `reduce()`
- Quando a função é usada apenas uma vez

Lambda com filter e reduce

Exemplo com filter:

```
1 num = range(1, 10)
2 for i in range(2, 4):
3     num = filter(lambda x: x % i == 0, num)
4 print(list(num))    # Numeros divisiveis por 2 ou 3
```

Exemplo com reduce:

```
1 from functools import reduce
2 lista = [1, 2, 3, 4]
3 a = reduce(lambda x, y: x + y, lista)
4 print(a)    # 10 = 1+2+3+4
```

Como funciona o reduce:

- 1 Aplica função aos dois primeiros elementos
- 2 Aplica função ao resultado e ao próximo elemento
- 3 Continua até sobrar um único elemento

Visualização:

```
1 ((1 + 2) + 3) + 4 = 10
```

Tratamento de Exceções

Erros detectados durante a execução são chamados de exceções.

```
1 while True:
2     try:
3         x = int(input('Please enter a number: '))
4         break
5     except ValueError:
6         print('Not a valid number...')
```


Vários excepts:

```
1 import sys
2
3 try:
4     f = open('myfile.txt')
5     s = f.readline()
6     i = int(s.strip())
7 except IOError as e:
8     print('I/O error({0}): {1}'.format(e.errno, e.strerror))
9 except ValueError:
10    print('Could not convert data to an integer.')
11 except:
12    print('Unexpected error:', sys.exc_info()[0])
13    raise
14 else:
15    print('Success!', i)
```

Mudar a cor da saída ajuda a identificar problemas rapidamente.

```
1 from itertools import chain
2
3 m = 'A message'
4 colors = chain(range(30, 39), range(40, 47), range(90, 99))
5
6 for color in colors:
7     print('\033[%i%sm' % color, m, '\033[0m', color)
8
9 print('\033[1m', m, '\033[0m', 'bold')
10 print('\033[3m', m, '\033[0m', 'italics')
11 print('\033[4m', m, '\033[0m', 'underline')
```

Códigos de cor comuns:

Código	Cor
30	Preto
31	Vermelho
32	Verde
33	Amarelo
34	Azul
35	Magenta
36	Ciano
37	Branco

Métodos Mágicos

Métodos especiais cercados por duplo underline (`__init__`, `__add__`, etc.).

```
1 class Counter:
2     def __init__(self, name):
3         self.name = name
4         self.count = 0
5
6     def __iadd__(self, number):    # +=
7         self.count += number
8         return self
9
10    def __add__(self, other):      # +
11        tmp = Counter(self.name + "+" + other.name)
12        tmp.count = self.count + other.count
13        return tmp
14
15    def __str__(self):             # str()
16        return 'Counter ' + self.name + ', ' + str(self.count)
```

```
1 a = Counter('foo')
2 b = Counter('bar')
3 a += 2
4 b += 1
5 c = a + b
6 print(a)    # Counter foo 2
7 print(b)    # Counter bar 1
8 print(c)    # Counter foo+bar 3
```

Multiprocessamento

Dividir um trabalho em vários processos para execução paralela.

```
1 import multiprocessing as mp
2 import psutil
3 import time
4 import os, sys
5
6 MAX_WORKERS = max(mp.cpu_count() - 1, 1)
7 pool = mp.Pool(MAX_WORKERS)
8 lock = mp.Lock()
9 N = len(datasets)
10 pids = []
11
12 for i, d in enumerate(datasets):
13     txt = "Dataset %s/%s" % (i+1, N)
14     p = mp.Process(target=eventloop, args=[..., txt, lock])
15     p.start()
16     pids.append(p.pid)
```

```
1
2 native = len(pids)
3 os.system('setterm -cursor off')
4 while native > 0:
5     al = 0
6     for pid in pids:
7         if psutil.Process(pid).status() != psutil.STATUS_ZOMBIE:
8             al = al + 1
9     native = al
10    dt = time.time() - t0
11    lock.acquire()
12    sys.stdout.write("Processes running %s/%s" % (native, len(pids))
13                      )
14    sys.stdout.flush()
15    lock.release()
```

JSON em Python - Leitura

JSON (JavaScript Object Notation) é um formato leve de troca de dados.

Leitura de JSON:

```
1 import json
2
3 # Lendo de um arquivo
4 with open('dados.json', 'r') as arquivo:
5     dados = json.load(arquivo)
6
7 # Lendo de uma string
8 json_string = '{"nome": "Joao", "idade": 30, "cidade": "Sao Paulo"}'
9 dados = json.loads(json_string)
10
11 # Acessando os dados
12 print(dados['nome'])      # Joao
13 print(dados['idade'])     # 30
14 print(dados['cidade'])    # Sao Paulo
```


JSON em Python - Leitura

```
1 # JSON com listas
2 json_string = '[{"x": 1, "y": 2}, {"x": 3, "y": 4}]'
3 dados = json.loads(json_string)
4 for ponto in dados:
5     print(f"Ponto: ({ponto['x']}, {ponto['y']})")
```

Escrita de JSON:

```
1 import json
2 # Dicionario Python
3 dataset={
4     "namespace": "fardet-hd",
5     "core.group": "dune",
6     "core.run_type": "fardet-hd",
7     "core.application.version": "v10_20_09d01",
8     "core.application.name": "cafmaker",
9     "core.data_tier": "caf",
10    "core.file_type": "mc",
11    "core.format_variant": "caf",
12    "core.run_type": "fardet-hd",
13    "dune.campaign": "fdhd_he_2026a_caf",
14    "dune.config_file": "cafmaker_atmos_dune10kt_1x2x6_runreco -
15    nuenergy-nuangular_geov5.fcl"
16 }
```

JSON em Python - Escrita

```
1 # Escrevendo em um arquivo (formatado)
2 with open('caf.json', 'w') as file:
3     json.dump(dataset, file, indent=4, sort_keys=True)
4
5 # Convertendo para string
6 json_string = json.dumps(dataset, indent=2)
7 print(json_string)
8
9 # Tratando objetos nao serializaveis
10 from datetime import datetime
11 data_with_date = {"core.date": datetime.now()}
12 json_string = json.dumps(data_with_date, default=str)
```